

CLAUDE CODE DEEP DIVE

Tutorial6

Source: Notes.md

1. First clone the repos from `https://github.com/mosh-hamedani/expense-tracker-starter` and `https://github.com/mosh-hamedani/helpdesk`. I will fix the bugs, refactor the code and add new features.
2. Claude code is an agentic coding tool. Basically, it is a coding agent. A coding agent can access to tools like write, read, bash, git etc. on its own and do actions.

Other coding agent are github co-pilot, windsurf, cursor etc.

Claude Pricing:

<https://claude.com/pricing#api> <https://claude.com/pricing#team-&-enterprise> <https://claude.com/pricing>

3. Run the commands: `ankit@MacBook-Air 25.Claude-Code-Deep-Dive % cd 7.Tutorial6` `ankit@MacBook-Air 7.Tutorial6 % mkdir playground` `ankit@MacBook-Air 7.Tutorial6 % cd playground` `ankit@MacBook-Air playground % git clone https://github.com/mosh-hamedani/expense-tracker-starter.git` Cloning into 'expense-tracker-starter'... remote: Enumerating objects: 18, done. remote: Counting objects: 100% (11/11), done. remote: Compressing objects: 100% (11/11), done. remote: Total 18 (delta 0), reused 0 (delta 0), pack-reused 7 (from 1) Receiving objects: 100% (18/18), 29.79 KiB | 14.90 MiB/s, done. `ankit@MacBook-Air playground % cd expense-tracker-starter` `ankit@MacBook-Air expense-tracker-starter %`
4. For multiline terminal, you can type `/terminal-setup` and then using `Shift+Enter`, you can use multiline terminal. It is better than put backslash `\` after each line.
5. Now run the bash command using exclamation mark `!` at the start as `! npm run dev` in claude terminal. Output will be `sh: vite: command not found`. The better but slower way is to ask `run this app` then claude will figure out and install the dependencies after giving the permissions. Now you can use `/tasks` command to see all the background tasks.
6. Creating Project Memory:

When you start a new project then run `/init` command to generate `agents.md` file and also generate the `claude.md` file in the current folder. It has project specific architecture, prompts and other things.

7. LLMs are non-deterministic and so it can't be predictable. So, everytime, output of `/init` is different.
8. You can use git commands in bash mode like `! git add . && git commit -m "chore: update" && git push origin main` or using source control in vscode or using claude terminal like `commit` but it consume some tokens and cost but generate a beautiful commit message.

9. To make terminal elegant you can ask `make the vscode terminal and claude terminal beautiful and elegant with colors and display of git branch or amny other features so make it more beautiful and elegant with modern look like uv package manager or any other modern beautiful and elegant terminal.`

10. Effective Prompting:

- Be specific and clear

So, instead of saying “Add Authentication”, say “Add JWT-Based Authentication to the login endpoint using the existing User model.”

- Give Context
- Be concise

For example:

`In @src/App.jsx, total income and expenses are not calculated correctly. Fix it.`

then ask, `In transactions array, amount should be a number.`

11. we can select few lines of the code in any file and ask like `explain the selected code`.

For example:

In `App.jsx` file, select the line `date: new Date().toISOString().split('T')[0]` and then ask `explain the selected code`.

12. Using the Plan Mode:

It is useful to build new features.

For example:

Enable the plan mode and ask `add the ability to delete transactions`.

Now, when plan is made then it will ask few options like auto-accept edits, manually approve edits etc, So, select `Tell Claude what to change` and then put `add the confirmation dialogue box` in the plan mode itself for this option. Now, select auto-accept edits option.

13. Managing the Context:

Think Context Window as claude’s working memory. So, it is like claude can see at any given moment. Context Window can contain conversations, files, [claude.md](#) etc.

Now, put the `/context` command and see the content. Roughly Token is 3/4th of a word.

System tools are like bash command tools, read, write, git etc related tools.

Now, we have `/compact` command which we use when context window gets full and we are working on a related task and we don’t want to lose history or context. Claude will put the summary in context and clear the original context after the execution of this command.

So, after executing `/compact` and then `/context`, we can see less consumption of tokens as compared to previous output of `/context`.

`/clear` is used to clear the context and conversation history. So, to clear unrelated task, use it.

Now, again if we use `/clear` and then `/context`, we get different output. (Why?)

14. Claude automatically compacts at ~70%.

15. Tracking cost and usage:

`/cost` for API usage and `/usage` for Claude subscription plan.

16. Cost Management:

Use

- `/clear`
- `/compact`
- Keep [Claude.md](#) file lean and relevant because this file goes to model for each request.